

Chapter Overview

Chapter 11 covers concurrency and locking in ShopVerse: Java 21 virtual threads, synchronized blocks, ReentrantLock, StampedLock, atomic variables, thread-safe collections, CompletableFuture, optimistic vs pessimistic DB locking, Redis distributed locks (Redisson), deadlock detection, and concurrency testing strategies.

Sub-Chapter	Topic
11-01	Java 21 Virtual Threads – M:N scheduling, Project Loom
11-02	Synchronized & ReentrantLock – intrinsic vs explicit locking
11-03	StampedLock & ReadWriteLock – read-optimised patterns
11-04	Atomic Variables – AtomicInteger, LongAdder, AtomicReference
11-05	Thread-Safe Collections – ConcurrentHashMap, CopyOnWriteArrayList
11-06	CompletableFuture & Structured Concurrency (Java 21)
11-07	Optimistic Locking – @Version, ObjectOptimisticLockingFailureException
11-08	Pessimistic Locking – SELECT FOR UPDATE, LockModeType
11-09	Distributed Locking – Redis Redisson RLock, Redlock algorithm
11-10	Deadlock Detection & Prevention
11-11	Concurrency Testing – CountdownLatch stress tests
11-12	Spring @Async & ThreadPoolTaskExecutor sizing

Aspect	Platform Thread	Virtual Thread (Java 21)
OS mapping	1:1 OS thread	M:N – many VTs per carrier thread
Creation cost	~1 MB stack; expensive	Lightweight – few KB on heap
Blocking I/O	Blocks the OS thread (wastes CPU)	Unmounts from carrier; carrier stays free
Max throughput	Limited by OS thread count (~few thousand)	Scalable – millions of concurrent I/O tasks
CPU-bound work	Native performance	No benefit; still needs carrier thread
Spring Boot 3.2	Default platform thread executor	spring.threads.virtual.enabled=true

```
// Enable virtual threads in Spring Boot 3.2+
spring.threads.virtual.enabled=true
// Manual executor bean
@Bean(APPLICATION_TASK_EXECUTOR_BEAN_NAME)
AsyncTaskExecutor applicationTaskExecutor() {
    return new TaskExecutorAdapter(
        Executors.newVirtualThreadPerTaskExecutor());
}
```

Concurrency Model Hierarchy

Java 21 Concurrency – overview

Platform Threads – backed by OS threads (JVM default)

`Executors.newFixedThreadPool(n)` – classic bounded pool

`ForkJoinPool.commonPool()` – parallel streams, `CompletableFuture`

Virtual Threads (Project Loom) – carrier thread pool underneath

`Executors.newVirtualThreadPerTaskExecutor()` – one VT per task

Unmounts from carrier on blocking I/O; carrier picks up next VT

Structured Concurrency (Java 21 preview)

`StructuredTaskScope.ShutdownOnFailure` – cancel all if any fails

`StructuredTaskScope.ShutdownOnSuccess` – cancel rest when first wins

Intrinsic Lock (synchronized)

```
public synchronized void decrementStock(Long productId, int qty) {
    Product p = products.get(productId);
    if (p.getStock() < qty) throw new InsufficientStockException();
    p.setStock(p.getStock() - qty);
}
```

ReentrantLock – explicit control + tryLock

```
private final ReentrantLock lock = new ReentrantLock(true); // fair
public void decrementStock(Long productId, int qty) {
    lock.lock();
    try {
        Product p = products.get(productId);
        if (p.getStock() < qty) throw new InsufficientStockException();
        p.setStock(p.getStock() - qty);
    } finally {
        lock.unlock(); // always in finally block
    }
}

// Non-blocking attempt with timeout
if (lock.tryLock(500, TimeUnit.MILLISECONDS)) {
    try { /* work */ } finally { lock.unlock(); }
} else { throw new LockTimeoutException("product " + productId); }
```

Feature	synchronized	ReentrantLock
Unlock	Automatic (end of block)	Manual – must be in finally
Interruptible wait	No	lockInterruptibly()
Try without blocking	No	tryLock()
Try with timeout	No	tryLock(time, unit)
Fairness	Not guaranteed	new ReentrantLock(true) – FIFO ordering
Multiple conditions	One per object (wait/notify)	Multiple via lock.newCondition()

```
// ReadWriteLock - concurrent reads, exclusive write
private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
private final Lock readLock = rwLock.readLock();
private final Lock writeLock = rwLock.writeLock();
public Product getProduct(Long id) {
    readLock.lock();
    try { return cache.get(id); }
    finally { readLock.unlock(); }
}
public void updateProduct(Product p) {
    writeLock.lock();
    try { cache.put(p.getId(), p); }
    finally { writeLock.unlock(); }
}

// StampedLock - optimistic read (no lock overhead if no writer)
private final StampedLock sl = new StampedLock();
public double readPrice(Long id) {
    long stamp = sl.tryOptimisticRead(); // no lock taken
    double price = cache.get(id).getPrice();
    if (!sl.validate(stamp)) { // was there a write since stamp?
        stamp = sl.readLock(); // fall back to proper read lock
        try { price = cache.get(id).getPrice(); }
        finally { sl.unlockRead(stamp); }
    }
    return price;
}
```

Lock	Concurrent Reads	Write Exclusive	Optimistic Read
synchronized	No – one thread only	Yes	No
ReentrantReadWriteLock	Yes – blocks on write	Yes only	No
StampedLock	Yes	Yes	Yes – try without locking; validate before use

```
// AtomicInteger - lock-free CAS (compare-and-swap)
AtomicInteger requestCount = new AtomicInteger(0);
requestCount.incrementAndGet();
requestCount.addAndGet(5);
requestCount.compareAndSet(10, 0); // if 10 then set to 0
// AtomicReference - atomic swap of immutable object
AtomicReference<PriceTable> ref = new AtomicReference<>();
ref.set(newPriceTable);
PriceTable current = ref.get();
// LongAdder - high-contention counter (splits into cells)
LongAdder pageViews = new LongAdder();
pageViews.increment(); // low contention even at high throughput
long total = pageViews.sum(); // approximate snapshot
```

Class	Operation	Best Use Case	Note
AtomicInteger/Long	incrementAndGet, CAS	Request counters, sequential updates	Single value; fine under moderate contention
LongAdder	increment, sum	High-throughput metrics, page views	Page view stripping; sum() is eventual read
AtomicReference	get, set, CAS	Swap immutable configuration atomically	Reference should be immutable for correctness
AtomicStampedRef	CAS + version stamp	ABA problem prevention	Can track version in memory

Collection	Thread-Safety	Use Case in ShopVerse
HashMap	None – data corruption	Never in multithreaded context
Collections.synchronizedMap	Full lock on every op	Legacy; avoid – use ConcurrentHashMap
ConcurrentHashMap	Segment/bin-level locking; reads almost lock-free	Product cache, session store
CopyOnWriteArrayList	Write = full array copy; reads never block	Category list – read-heavy, rare writes
ConcurrentLinkedQueue	Lock-free CAS FIFO queue	Order processing pipeline hand-off
LinkedBlockingQueue	Bounded; put/take block on full/empty	Producer-consumer with back-pressure
ConcurrentSkipListMap	Lock-free sorted map $O(\log n)$	Price-sorted product map

```
// ConcurrentHashMap – atomic check-and-insert (no race condition)
ConcurrentHashMap<Long, ProductCache> cache = new ConcurrentHashMap<>();
cache.computeIfAbsent(productId, id -> loadFromDB(id));
// Atomic conditional update
cache.compute(productId, (id, existing) ->
existing == null ? new ProductCache(product) : existing.update(product));
```

```
// CompletableFuture - async fan-out pipeline
CompletableFuture<ProductDetailDto> future =
CompletableFuture
    .supplyAsync(() -> productRepo.findById(id).orElseThrow(), vtExecutor)
    .thenCombineAsync(
        CompletableFuture.supplyAsync(() -> inventoryService.getStock(id), vtExecutor),
        (product, stock) -> new ProductDetailDto(product, stock))
    .orTimeout(3, TimeUnit.SECONDS)
    .exceptionally(ex -> ProductDetailDto.fallback(id));

// Structured Concurrency (Java 21) - cleaner fan-out with cancel
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Future<Product> p = scope.fork(() -> productRepo.findById(id).orElseThrow());
    Future<Integer> st = scope.fork(() -> inventoryService.getStock(id));
    Future<List<Review>> r = scope.fork(() -> reviewService.getTop5(id));
    scope.join().throwIfFailed();
    return new ProductDetailDto(p.get(), st.get(), r.get());
}
```

Pattern	Use When	Cancel On Failure?
thenCombine(f1,f2)	Two independent futures, merge results	Yes – one failure does not cancel other
allOf(futures)	Wait for all to complete	No
anyOf(futures)	First to complete wins	Remaining continue
ShutdownOnFailure	All tasks must succeed	Yes – cancels all on first failure
ShutdownOnSuccess	Any one success is enough	Yes – cancels rest on first success

```

@Entity public class Product {
    @Id @GeneratedValue private Long id;
    private int stock;
    @Version private Long version; // Hibernate manages this field
}

// Hibernate generated SQL on save:
// UPDATE products SET stock=?, version=? WHERE id=? AND version=?
// If 0 rows affected: another thread updated first
// Throws: ObjectOptimisticLockingFailureException
@Retryable(retryFor = OptimisticLockingFailureException.class, maxAttempts = 3)
@Transactional
public void decrementStock(Long productId, int qty) {
    Product p = productRepo.findById(productId).orElseThrow();
    if (p.getStock() < qty) throw new InsufficientStockException();
    p.setStock(p.getStock() - qty);
    productRepo.save(p); // may throw OptimisticLockingFailureException
}

```

Step	Thread A	Thread B
1 – Both read	Product version=1	Product version=1
2 – A updates first	UPDATE WHERE version=1 → 1 row → success → version=2	Still sees version=1
3 – B tries to update	—	UPDATE WHERE version=1 → 0 rows → exception
4 – B retries	—	Re-read version=2, update → success


```

@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT p FROM Product p WHERE p.id = :id")
Optional<Product> findByIdForUpdate(@Param("id") Long id);
@Transactional(timeout = 5)
public OrderItem reserveStock(Long productId, int qty) {
// SELECT ... FOR UPDATE - other transactions block until this commits
Product p = productRepo.findByIdForUpdate(productId).orElseThrow();
if (p.getStock() < qty) throw new InsufficientStockException();
p.setStock(p.getStock() - qty);
return orderItemRepo.save(new OrderItem(p, qty));
}

```

Lock Mode	SQL Generated	Other Transactions Blocked	Use Case
PESSIMISTIC_READ	SELECT ... FOR SHARE	Write only	Prevent dirty read; read-rarely-written data
PESSIMISTIC_WRITE	SELECT ... FOR UPDATE	Read and write	Flash sale; stock decrement; financial ops

	Optimistic	Pessimistic
Contention	Low – retry rarely needed	High – frequent conflicts expected
DB overhead	No lock row held	Lock held for transaction duration
Deadlock risk	None (no lock held)	Yes – order locks consistently
Best for	Most CRUD; rare concurrent writes	Flash sales, inventory, payments

```

@Service public class FlashSaleService {
    private final RedissonClient redisson;
    public void purchase(Long productId, Long userId) {
        RLock lock = redisson.getLock("flash:" + productId);
        boolean acquired = false;
        try {
            // waitTime=500ms (timeout), leaseTime=5s (auto-release on JVM crash)
            acquired = lock.tryLock(500, 5000, TimeUnit.MILLISECONDS);
            if (!acquired) throw new LockNotAvailableException();
            decrementStock(productId);
            createOrder(userId, productId);
        } finally {
            if (acquired && lock.isHeldByCurrentThread())
                lock.unlock();
        }
    }
}

```

Lock Type	Nodes Required	Guarantees	ShopVerse Use
RLock (single Redis)	1	Mutual exclusion (not fault-tolerant)	Dev / single-node Redis
RedLock	3+ independent Redis nodes	Majority quorum; survives minority failure	Prod HA distributed lock
RReadWriteLock	1 or cluster	Distributed read-write lock semantics	Shared config cache invalidation
RSemaphore	1 or cluster	N permits – bounded concurrency	API rate limiting per endpoint

Coffman Conditions (all 4 must hold for deadlock)

Deadlock requires ALL 4 conditions simultaneously

1. Mutual Exclusion – resource held exclusively (a lock)
2. Hold and Wait – holds one lock while waiting for another
3. No Preemption – lock cannot be forcefully taken away
4. Circular Wait – Thread A waits for B; B waits for A

Prevention – break any one condition

- Lock Ordering – always acquire in same global order (breaks circular wait)
- tryLock with timeout – release held lock on timeout (breaks hold-and-wait)
- Consolidate – use one lock for related resources (breaks hold-and-wait)

```
// Deadlock prevention: consistent lock ordering (smaller ID first)
public void transfer(Account from, Account to, BigDecimal amount) {
    Account first = from.getId().compareTo(to.getId()) < 0 ? from : to;
    Account second = from.getId().compareTo(to.getId()) < 0 ? to : from;
    synchronized (first) {
        synchronized (second) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}

// Detection: JVM thread dump
jstack <pid> # Look for: "Found one Java-level deadlock"
```

```
// CountdownLatch stress test - 100 virtual threads, simultaneous start
@Test void testConcurrentStockDecrement() throws Exception {
    int N = 100;
    CountdownLatch ready = new CountdownLatch(N);
    CountdownLatch start = new CountdownLatch(1);
    CountdownLatch finish = new CountdownLatch(N);
    AtomicInteger success = new AtomicInteger(0);
    AtomicInteger fail = new AtomicInteger(0);
    for (int i = 0; i < N; i++) {
        Thread.ofVirtual().start(() -> {
            try {
                ready.countDown();
                start.await(); // all threads start simultaneously
                stockService.decrement(PRODUCT_ID, 1);
                success.incrementAndGet();
            } catch (Exception e) {
                fail.incrementAndGet();
            } finally { finish.countDown(); }
        });
    }
    ready.await(); start.countDown(); finish.await(10, SECONDS);
    assertThat(success.get() + fail.get()).isEqualTo(N);
    assertThat(stockService.getStock(PRODUCT_ID)).isGreaterThanOrEqualTo(0);
}
```

```

@Configuration @EnableAsync
public class AsyncConfig {
    @Bean("notificationExecutor")
    public Executor notificationExecutor() {
        ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
        e.setCorePoolSize(10); // always-alive threads
        e.setMaxPoolSize(50); // burst capacity
        e.setQueueCapacity(500); // buffer before spawning max
        e.setThreadNamePrefix("notify-");
        e.setRejectedExecutionHandler(new CallerRunsPolicy());
        e.initialize();
        return e;
    }
}

@Async("notificationExecutor")
public CompletableFuture<Void> sendOrderConfirmation(Order order) {
    emailService.sendHtml(order.getEmail(), buildTemplate(order));
    return CompletableFuture.completedFuture(null);
}

```

ThreadPool Setting	Wrong Value Effect	Recommendation
corePoolSize too small	Queue fills early; latency spike	Match steady-state concurrency
maxPoolSize too large	Context-switch overhead; OOM risk	2-4× corePoolSize; monitor queue
queueCapacity=0 (SynchronousQueue)	Unbounded queue; per request up to max CPU-bound; avoid for I/O-bound	Prefer
CallerRunsPolicy	Caller thread runs task (back-pressure)	Preferred – prevents task drops
AbortPolicy (default)	Throws RejectedExecutionException	Only if caller can handle rejection

■ Do

- ✓ Enable virtual threads in Spring Boot 3.2 for I/O-bound services – eliminates thread starvation.
- ✓ Always release locks in finally blocks to prevent lock leak on exceptions.
- ✓ Use `ReentrantLock.tryLock(timeout)` to avoid indefinite blocking – return `LockTimeoutException`.
- ✓ Use `@Version` (optimistic) for most entities; reserve `PESSIMISTIC_WRITE` for flash-sale stock.
- ✓ Acquire multiple locks in a consistent global order (e.g. always lower ID first) to prevent deadlock.
- ✓ Use `LongAdder` instead of `AtomicLong` for high-contention counters (page views, order count).
- ✓ Set Redisson `leaseTime` on distributed lock – auto-releases if the holding JVM crashes.
- ✓ Stress test with `CountDownLatch`: 100+ virtual threads simultaneously – assert stock never goes negative.
- ✓ Use `ConcurrentHashMap.computeIfAbsent` for cache population – atomic check-and-insert.
- ✓ Use `CallerRunsPolicy` on thread pools – provides back-pressure instead of dropping tasks.

■ Anti-Patterns

- Using `HashMap` in multithreaded code – data corruption; use `ConcurrentHashMap`.
- Nested synchronized blocks acquired in different orders in different methods – deadlock.
- Holding a lock during long I/O (DB call, HTTP) – serialises all threads unnecessarily.
- Forgetting `lock.unlock()` outside finally block – lock never released on exception.
- Using `Thread.sleep()` for lock retry – thundering herd; use exponential back-off.
- No `leaseTime` on distributed lock – if JVM crashes, lock held indefinitely.
- Pessimistic lock on low-contention data – serialises requests with no benefit.

Sub-Chapter	Key Point	One-Liner
11-01 Virtual Threads	True scheduling	Unmounts on blocking I/O; scales to millions concurrent tasks
11-02 ReentrantLock	tryLock() in finally	tryLock(timeout) prevents indefinite blocking
11-03 StampedLock	Optimistic read stamp	validate(stamp) – if false, fall back to readLock
11-04 AtomicCAS	CAS lock-free	LongAdder beats AtomicLong under high contention
11-05 ConcurrentHashMap	Per-Header lock	computeIfAbsent is atomic check-and-insert
11-06 CompletableFuture	thenCombine, allOf	ShutdownOnFailure in Java 21 – cleaner fan-out
11-07 OptimisticVersion	@Version on entity	0 rows updated -> exception -> @Retryable
11-08 PessimisticSELECT FOR UPDATE	SELECT FOR UPDATE	Blocks all readers+writers; use for flash sale stock
11-09 Redisson	tryLock(wait, lease)	leaseTime auto-releases lock on JVM crash
11-10 Deadlock	Consistent lock order	Always lock lower-ID resource first
11-11 Stress Test	CountDownLatch	All threads start simultaneously; assert non-negative stock
11-12 @Async	ThreadPoolTaskExecutor	CallerRunsPolicy = back-pressure; never drop tasks

Interview Quick-Check

Question	Concise Answer
Optimistic vs Pessimistic?	Optimistic: @Version, no DB lock, retry on conflict. Pessimistic: SELECT FOR UPDATE, holds lock, blocks others.
Virtual threads vs platform?	Platform: 1:1 OS thread (limited). Virtual: M:N, unmounts on blocking I/O, scales to millions. Enable with VM flag.
How to prevent deadlock?	Consistent lock ordering (smaller ID first). tryLock with timeout. Minimise lock scope. Detect with jstack.
LongAdder vs AtomicLong?	High contention: AtomicLong retries CAS on collision. LongAdder stripes into per-cell counters; sum at commit.
What is Redlock?	Distributed lock across 3+ independent Redis nodes. Acquired when majority grant it. Tolerates min 2 nodes.